Date: -10/10/26

CO-1 Assessment-1

APPLICATIONS BASED QUESTIONS

Course Code: -DSA 0308

Course Name: -Natural Language Processing

Name: -A.V. Ameya **Reg no: -192472036**

CASE STUDY 1:-Resume information extraction using regular expressions

Aim:

To extract important details such as name, email, mobile number, and experience from a resume using Python Regular Expressions.

Information:

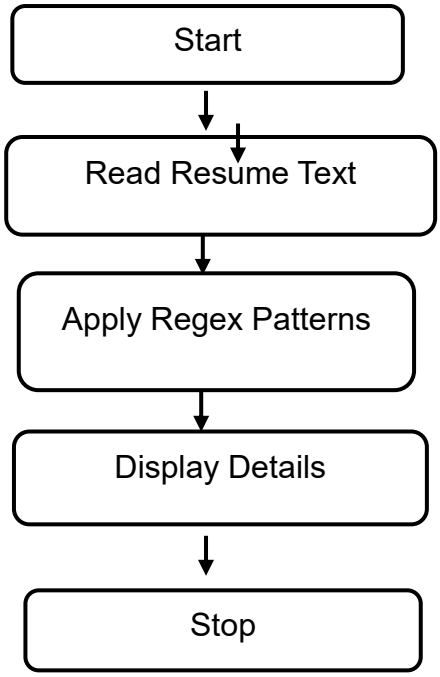
Regular Expressions (Regex) are used to search and extract specific patterns from text. The re module in Python helps identify email addresses, phone numbers, and other details.

A company receives hundreds of resumes from job applicants. Manually checking every resume to find details such as email addresses, mobile numbers, and experience takes a lot of time. To solve this problem, a Python program using Regular Expressions (Regex) can automatically extract important information from resume text. The re module searches for specific patterns, such as email addresses and 10-digit mobile numbers. This system reduces manual work and helps recruiters collect candidate information quickly.

Algorithm:

1. Start.
2. Enter registration number and mobile number.
3. Check both using Regex.
4. Display whether the details are valid.
5. Stop.

Flowchart:



Code:

```
import re

s = """Name: Ameya

Email: ameya@gmail.com

Mobile: 9876543210
```

Skills: Python

Experience: 3 years"""

print("Name:", re.search(r"Name: (.*)", s).group(1))

print("Email:", re.findall(r"[\w.-]+@[\w.-]+\.\w+", s))

print("Mobile:", re.findall(r"\b\d{10}\b", s))

y = int(re.search(r"Experience: (\d+)", s).group(1))

print("Experience:", y, "years")

print("Candidate Shortlisted" if y >= 2 and "Python" in s else "Not Shortlisted")



Conclusion :-

Regular Expressions help extract important information from resumes, such as email addresses and mobile numbers. This reduces manual work and saves recruiters time.

CASE STUDY 2: Product Search System Using Regular Expressions

Aim:-

To search products using Regular Expressions.

Information:

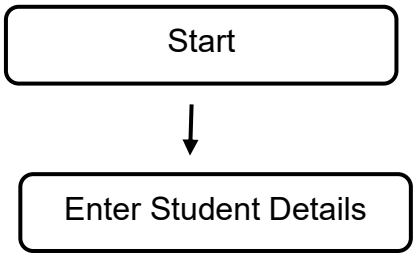
The system searches product names using keywords and ignores uppercase and lowercase differences. It displays matching products and their total count.

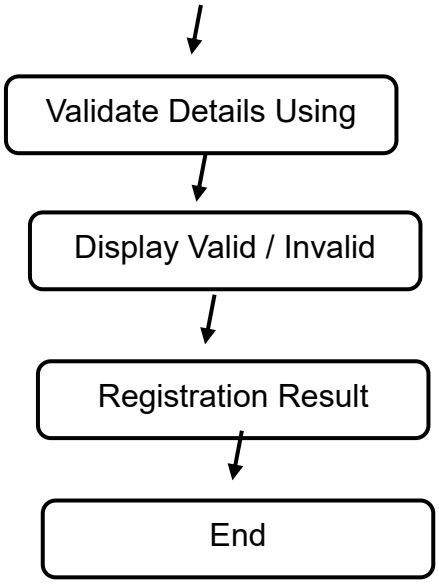
An online shopping website contains many products, making it difficult for customers to find a particular item quickly. A product search system using Regular Expressions helps users search for products based on keywords. For example, when a customer searches for “Apple,” the system identifies matching products such as Apple iPhone and Apple Watch. Python's re module compares the keyword with product names and displays the matching results. This makes product searching faster and more convenient.

Algorithm:

1. Start.
2. Store product names.
3. Enter a search keyword.
4. Search products using regex.
5. Display matching products and count.
6. Stop.

Flow chart:-





Code:-

```
import re

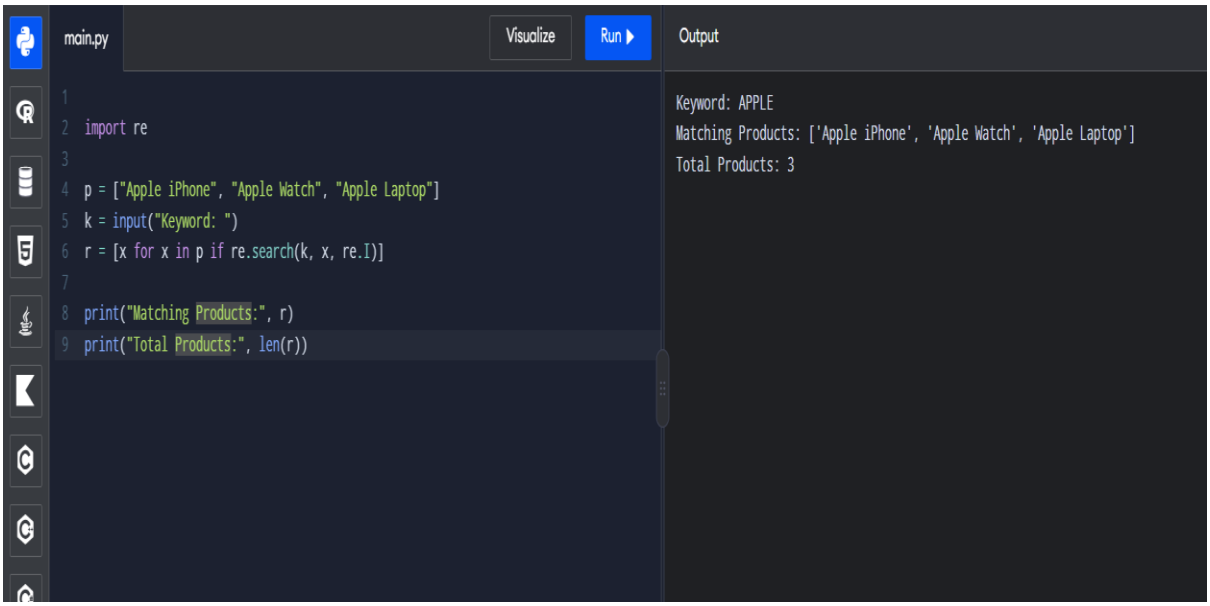
p = ["Apple iPhone", "Apple Watch", "Apple Laptop"]

k = input("Keyword: ")

r = [x for x in p if re.search(k, x, re.I)]

print("Matching Products:", r)

print("Total Products:", len(r))
```



Conclusion

Regular Expressions help users find products quickly using keywords. This makes product searching easier and improves the shopping experience.

CASE STUDY 3:- Case Study 3: University Registration System Using Regular Expressions

Aim:

To validate student registration details using Regular Expressions.

Information:

The system checks the student's register number, email, course code, semester and mobile number. It displays whether the details are valid and registration is successful.

A university registration portal collects student details such as registration numbers and mobile numbers. Incorrect information may cause problems during admission and record management. To prevent this, Regular Expressions can be used to validate the entered details. The system checks whether the registration number follows the required format and whether the mobile number contains exactly 10 digits. If the details match the specified patterns, they are accepted; otherwise, an error is displayed. This improves the accuracy of student registration and reduces invalid entries.

Algorithm:

1. Start.

2. Enter student details.
3. Validate each field using regex.
4. Display valid or invalid messages.
5. Display registration status.
6. Stop.

Code:-

```
import re

reg = input("Register Number: ")

email = input("Email: ")

mobile = input("Mobile: ")

a = bool(re.fullmatch(r"\d{12}", reg))

b = bool(re.fullmatch(r"[\w.-]+@[\w.-]+\.edu\.in", email))

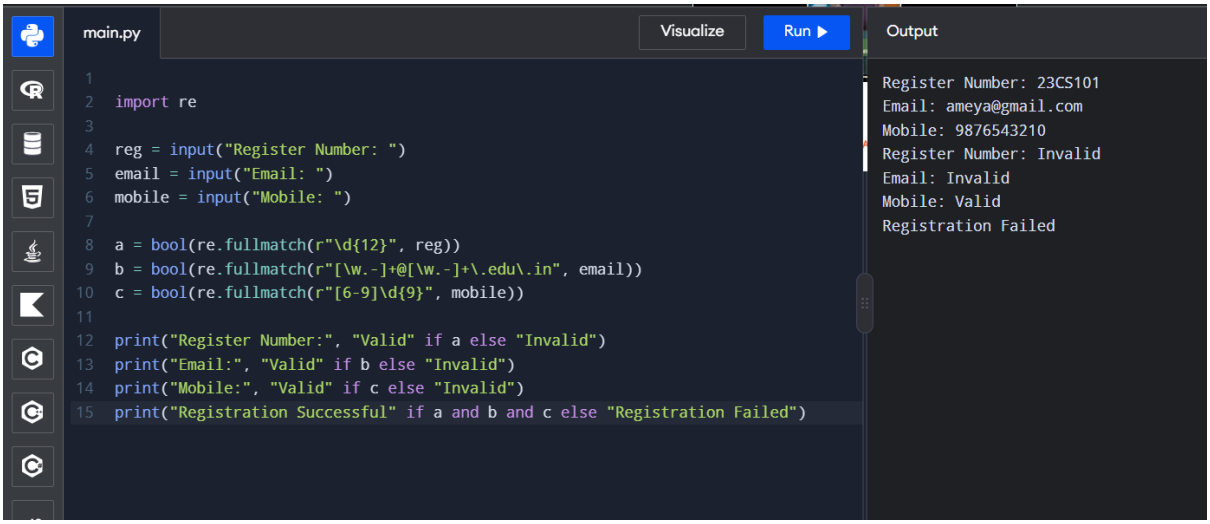
c = bool(re.fullmatch(r"[6-9]\d{9}", mobile))

print("Register Number:", "Valid" if a else "Invalid")

print("Email:", "Valid" if b else "Invalid")

print("Mobile:", "Valid" if c else "Invalid")

print("Registration Successful" if a and b and c else "Registration
Failed")
```



Conclusion –

Regular Expressions help validate student registration numbers and mobile numbers. This reduces incorrect entries and improves the accuracy of university records.